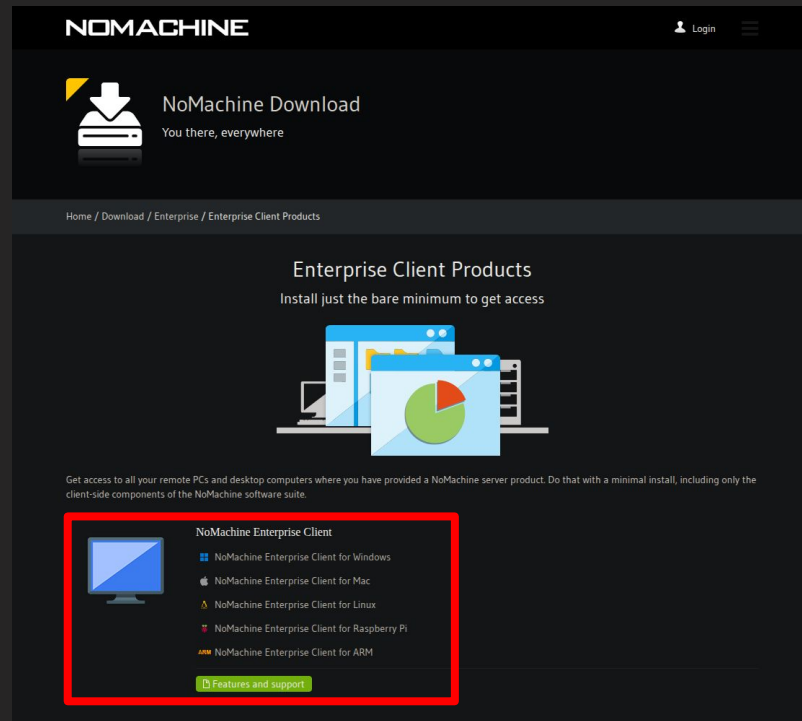
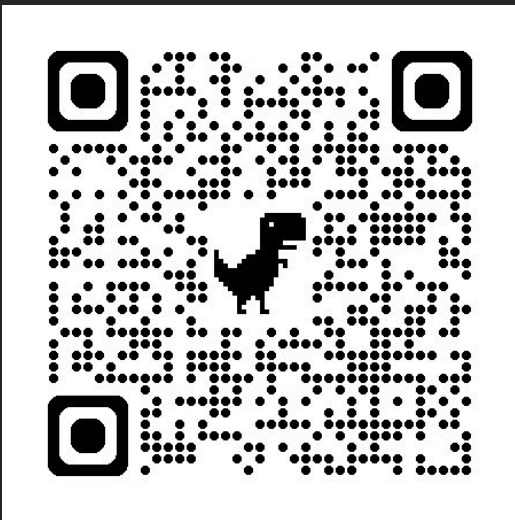


Required Software

Download **NoMachine Enterprise Client** on your laptop





Design Your Own CPU

*Session 3 – Optimisation and Bare-Metal
Programming*



Session 1 – Intro to SystemVerilog

Session 2 – The single-cycle CPU

Session 3 – Optimisation and bare-metal programming

About Me

- Part IV Electronic Engineering with Computer Systems
- *IP: A Universal Superscalar Processor Design for MIPS and RISC-V*
- *GDP: Verify a RISC-V Core in 10 Weeks*
- R&D Intern at Codaip (Fabless RISC-V IP firm)

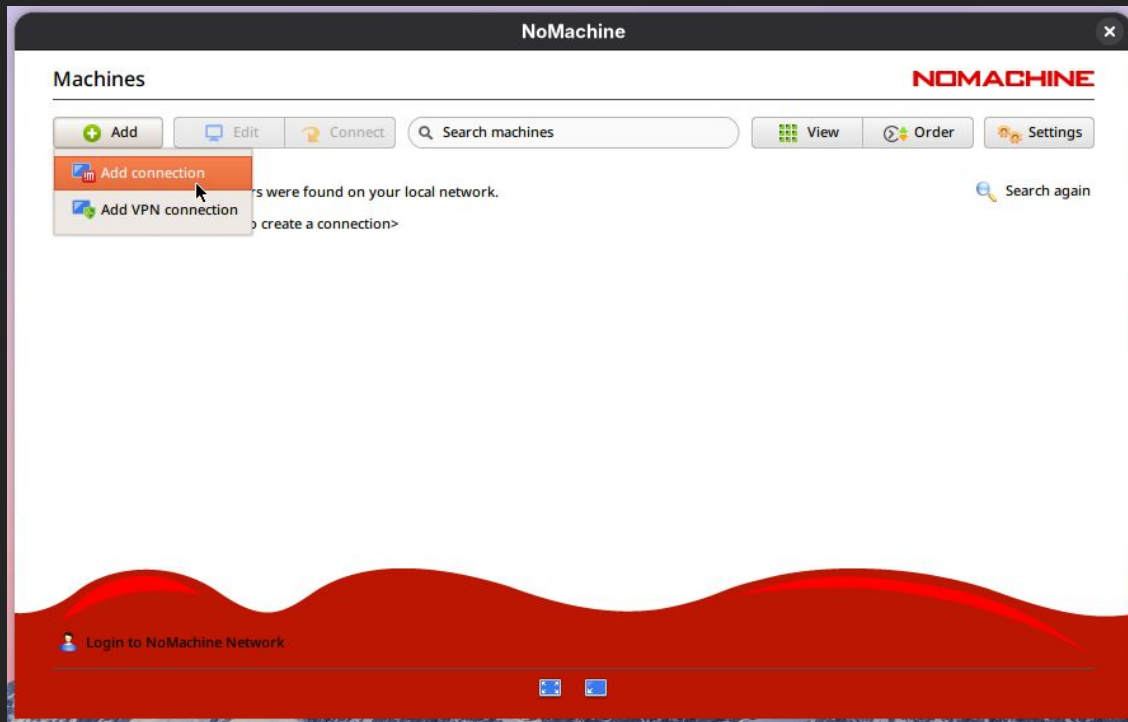
Reasons for Attending These Workshops

- You're interested in doing a digital design focused IP or GDP
- You want an internship in digital design
- You want to learn or have a refresher on SystemVerilog
- You want to know how digital systems are designed
- For fun

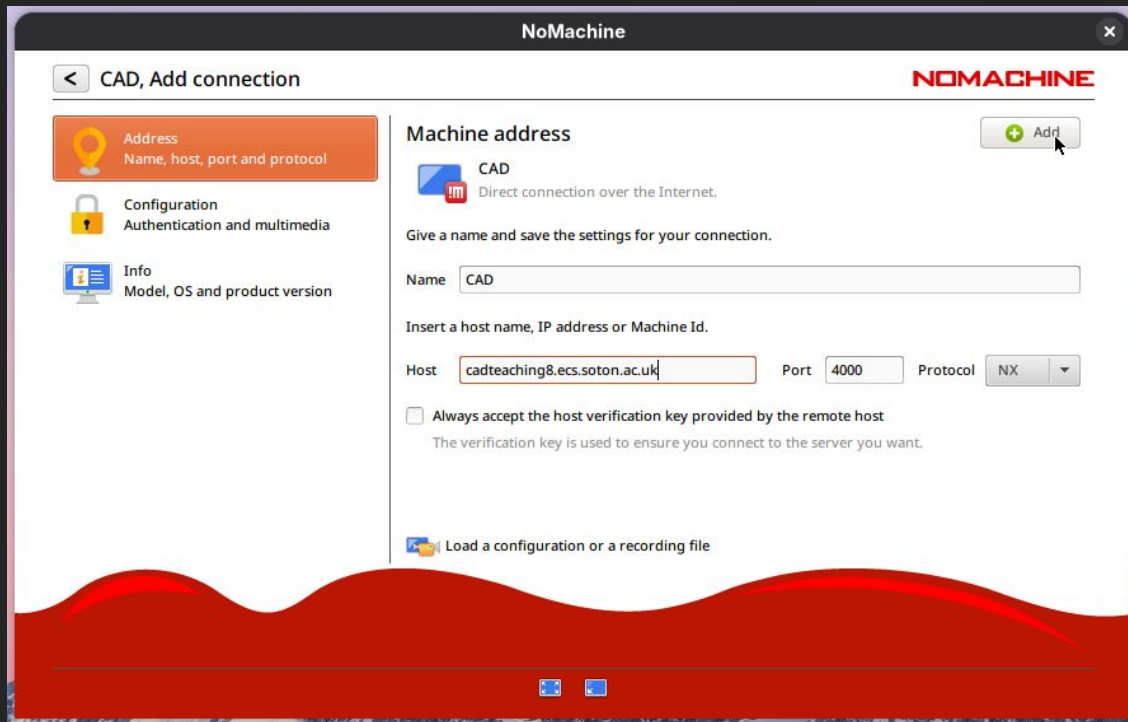


Setup

Setup (1)



Setup (2)



Setup (3)



Setup (4)



Setup (5)

- **If you were here last week:**
 - **Go to the folder we were working in**
 - `git pull`
- *Otherwise*, in your terminal, clone this Git repository:

<https://github.com/bhart17/design-your-own-cpu>

- `git clone https://github.com/bhart17/design-your-own-cpu.git`
- `cd design-your-own-cpu`



Recap

Overall: Designing digital systems

- We want to model systems that take in a number of binary inputs
- They compute outputs as a function of these inputs **and** the state of the system using boolean logic
- State is updated on a clock edge

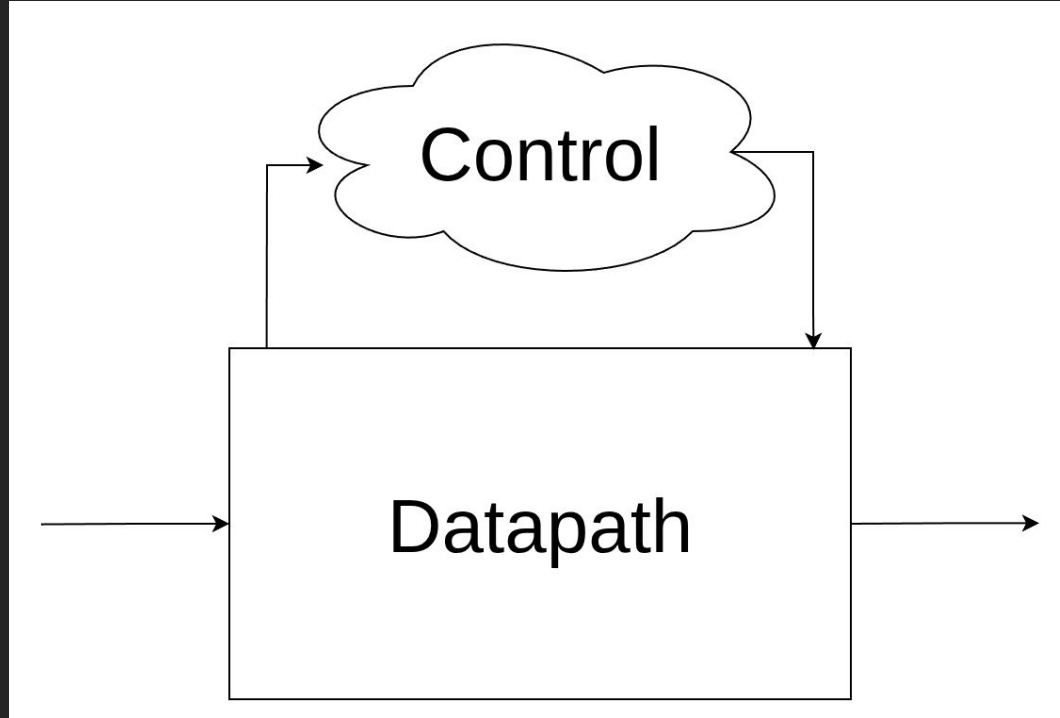
Levels of Abstraction for a Digital IC

1. Specification
2. Block diagram
3. Hardware description language (such as SystemVerilog)
4. Gate-level circuit (*Generic cells*)
5. Transistor-level circuit (*Specific cells*)
 - 5.1. Transistor circuit diagram
 - 5.2. Stick diagram
 - 5.3. Mask diagram

Hardware Description Languages

- Aiming to describe the logic between an input and output in a formal but high-level way
- Readable by humans but understandable to computers
- Historically called Register Transfer Layer – describing how data flows between *registers*
- Not programming languages!

CPUs



RISC-V

- Open-source *Instruction Set Architecture*
- Defines the set of functions that our CPU needs to perform and how those instructions are encoded
- We have freedom to implement those instructions in whatever architecture we want

Memory Mapped Peripherals

- CPUs need to interact with inputs and outputs
- Integrating specific I/O into every CPU is not trivial
- Solution: access all data via the data memory interface
- Some sections of memory are not ROM or RAM – they are any generic I/O, *mapped* as a memory address



Bare-Metal Programming

Bare-metal versus Hosted

- In a *hosted* environment like your laptop, an operating system manages the user programs you want to execute
- In our system, there is no kernel – we have to write one
- No access to most standard libraries
 - No `printf()`

Runtimes

- Languages need a runtime that sets up the environment and handles required tasks (if any)
- GCC would link your `C_main()` with a `_start` entrypoint in a hosted system, but we must bring our own in this environment



1. Hello World!

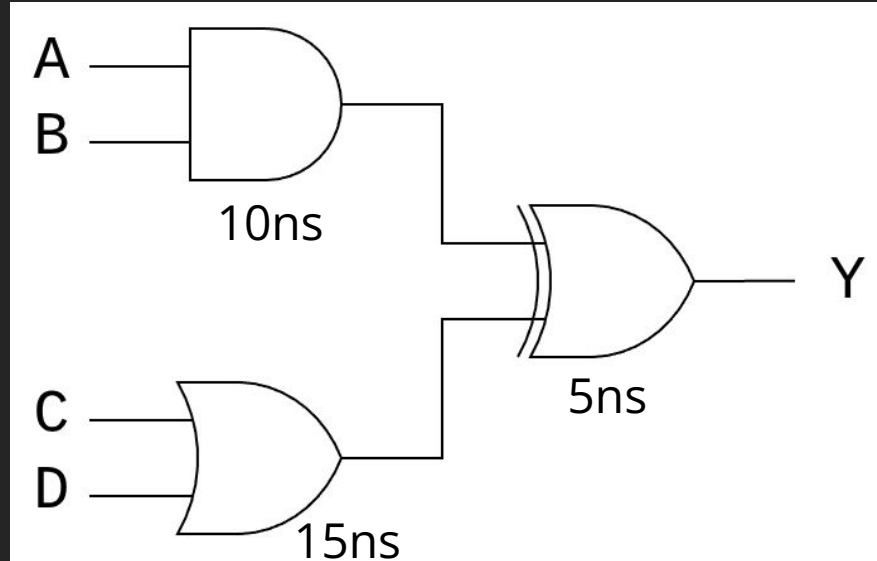
- Write bare-metal C to interact with our memory-mapped peripherals
- Compile using GCC



Pipelining

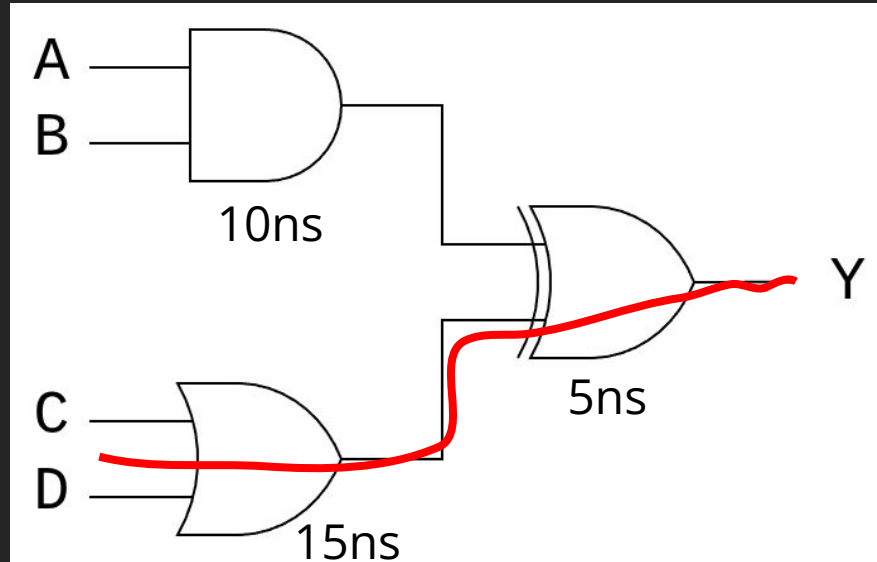
Delays

•



The Critical Path (1)

-



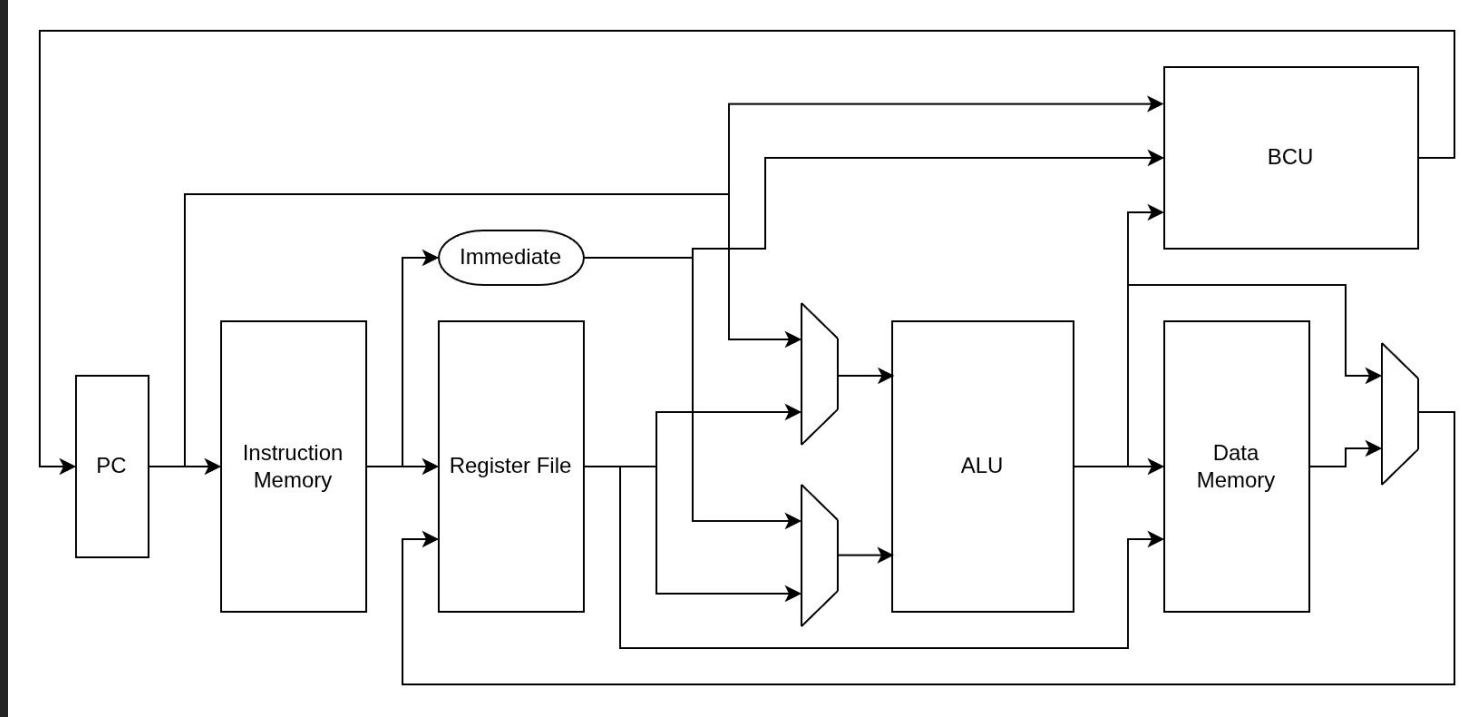
The Critical Path (2)

- Need to reduce the longest combinatorial path between two flip-flops
- This will allow us to run our IC at a faster frequency – better *throughput*
- We can do this by adding more registers in certain places

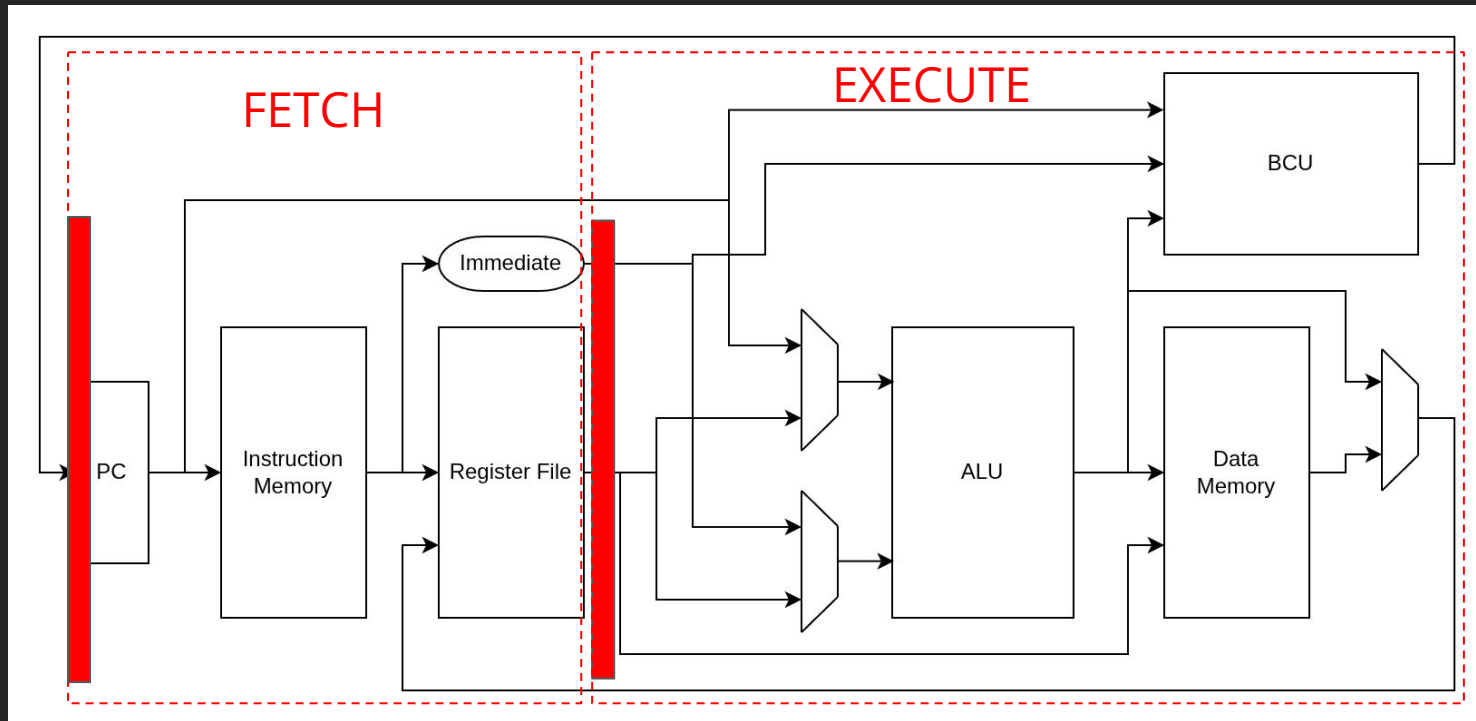
Pipelining

- Breaking up a long combinatorial path into multiple stages is called *pipelining*
- On each clock cycle, the intermediate result in one stage of the pipeline is stored in a register at the start of the next pipeline stage

Original Datapath



Simple Pipelined Datapath



2. Pipeline Registers

- Add pipeline registers between the Fetch and Execute stages
- First, add the PC and instruction registers
- Then, add the data registers

Hazards

- Now we have added pipeline stages, some combinations of instructions will not execute properly
- Data Hazard: an instruction uses a result of a previous instruction
- Structural Hazard: two instructions need the same resource
- Control Hazard: a branch has meant that a previous instruction fetch was incorrect

Forwarding and Flushes

- We can fix data hazards with *forwarding*: instead of taking the value as decoded, *forward* the newer value to where it is needed
- Control hazards can be fixed with *pipeline flushes*: if we fetched the wrong instruction, *flush* the pipeline when the branch resolves

3. Exe-Fetch Forwarding

- Add a forwarding path from the Execute stage to the decode of the Fetch stage
- Do this via the registerfile – make it *transparent*

4. Branch Flushes

- Add a condition to the pipeline registers to flush the current instruction if a branch was taken
- A `branch_taken` flag is provided from the BCU



Further Optimisations

More Pipeline Stages

- Adding more pipeline stages can reduce our critical path even further
- More hazards are introduced and branch flushes are more expensive
- Modern high-performance CPUs are likely to have 8 pipeline stages or more, and have a variable number depending on the instruction

Multiple Pipelines (Superscalar)

- We can also parallelise our pipelines
- Duplicate all of the hardware for a pipeline and fetch two instructions per clock cycle
- Even more hazards
- Allows us to achieve an IPC of greater than 1

Branch Prediction

- Instead of blindly assuming the branch wasn't taken and continuing sequentially, we can predict the outcome of the branch
- Decrease the number of times a pipeline must be flushed and hence increase performance

Out-of-Order Execution

- Modern cores do not execute instructions in the order of the program
- A memory access could delay the core by hundreds of cycles, while instructions after don't even depend on it
- We can *speculatively* execute instructions without unresolved dependencies, possibly hundreds of instructions ahead
- Flushes are very expensive! Have to roll back CPU state to last *commit*

Thank you!

*Some materials and inspiration in this presentation sourced from Iain McNally's notes
(<https://www.southampton.ac.uk/~bim/>)*